SO-101 ピック＆プレース 第2版：2本指グリッパ × 摩擦把持

ROS 2 × Gazebo × LeRobot (ACT) による模倣学習の全工程（Google Colab）
Physical AI 講座 2026 応用編1 最終課題（第2版） ／ omnicampus アカウント名：maty0505

タスク	ランダムな位置にある赤いブロック（2.5 cm 角の剛体）を、SO-101 アームで掴んで青いトレイに入れる
第1版からの変更	① ハンドを、片側だけが動く純正ジョーから、**両側の指が開閉する平行2本指グリッパ**に変更 ② ブロックを指と衝突する**剛体**にし、固定などの補助を使わず**指で挟んだ摩擦力だけ**で運ぶ ③ 指の内側に**ラバーパッド**、指の本体を**10 段階のなだらかな形状**に
使用技術	ROS 2 Jazzy ／ Gazebo Harmonic（物理エンジン DART）／ LeRobot 0.6.1（ACT）／ PyTorch ／ Google Colab（T4 GPU）
AI の活用	① 模倣学習ポリシー ACT がカメラ画像と関節状態から行動を生成して制御　② 開発過程で生成 AI アシスタントをコード作成・デバッグの補助に使用
コード	https://github.com/pai-claude-maty0505/second_model

5 / 5	**60 / 65**	**1.447 → 0.065**	**5 / 10**
把持テスト（手本の動作） 摩擦だけで成功	手本デモの成功 （5,883 フレーム）	ACT 学習損失 （10,000 ステップ）	自律実行の成功 （時間的アンサンブル）

1. 目的

第1版では、指が閉じたときにブロックを手先に「固定」する補助（DetachableJoint）を使っていました。また、ブロックと指は衝突しない設定で、見た目では指がブロックに食い込んでいました。第2版では、実機に近い条件として、**剛体のブロックを2本の指で挟み、摩擦力だけで持ち上げて運ぶ**ことを目標にしました。

2. ハンドの設計

2.1 平行2本指グリッパ

- SO-101 の公式モデルから、可動ジョーと固定ジョーの部品を取り外しました。代わりに、掌・レールと、左右に平行移動する 2 本の指（直動関節 `finger_left` / `finger_right`）を URDF に追加しました。
- 2 本の指には常に同じ指令を送るので、**左右対称に開閉**します。開き幅は中心から片側 0〜35 mm です。
- 各指は力の上限付きの位置制御（最大 15 N）です。ブロックより内側を目標にすると、指がブロックに当たって止まり、その力で挟みます。
- 手先の基準点（TCP）は第1版と同じ位置なので、逆運動学は第1版のものをそのまま使えます。

2.2 ラバーパッドとなだらかな指形状

当初の指は単純な直方体で、形状の段階が少なく角ばっていました。そこで、次のように改良しました。

- **ラバーパッド**：各指の内側（ブロックに触れる面）に、厚さ 3 mm の黒いゴムパッドを付けました（長さ 36 mm、縁は丸め）。シミュレーション上は摩擦係数 5.0 の面として扱い、指の本体（樹脂）は 1.0 にしました。
- **なだらかな形状**：指の本体を、根元の厚さ 10 mm から先端の 6 mm まで、最初は 3 段階で細くしていました。これを **10 段階**に増やしてなだらかにし、先端は丸めました。見た目の形状だけを細かくし、衝突判定はパッドと本体の 2 つの箱のままにしています（接触計算を安定させるため）。

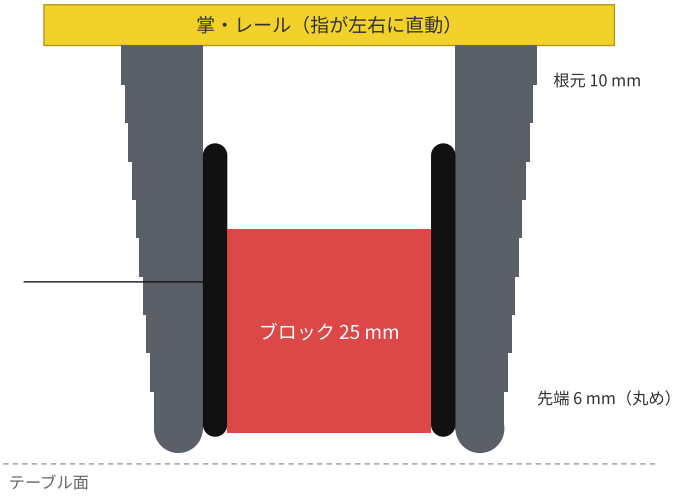


図1　指の断面（模式図）：10 段階のテーパーと内側のラバーパッド

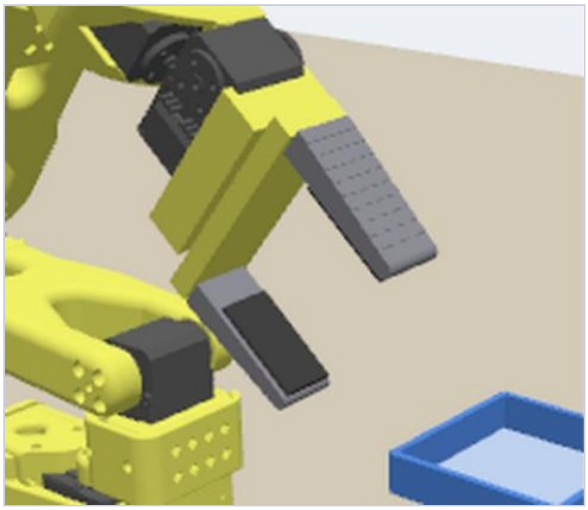


図2　Gazebo 上の指（灰色の本体と黒いラバーパッド）

> **注**：Gazebo の物理エンジン（DART）では、ゴムの柔らかさ（変形）までは再現できません。ラバーは「摩擦の大きい面」として表現しています。

2.3 把持の検証と調整

段階	設定	手本動作の結果	分かったこと・対策
初期	摩擦 1.5、把持力 8 N	3 回中 1 回成功	ブロックが指の間で滑り落ちる（持ち上げ中に 1.5 cm、運搬中に 3 cm）。トレイの上 8 cm には手首の可動範囲の都合で届かない
調整後	摩擦 3.0→5.0、把持力 15 N、持ち上げと運搬を遅く、トレイ上の高さ 6.5 cm	5 回中 4 回成功	運搬中の旋回で滑り出すことがまれにある
ラバーパッド	**パッド摩擦 5.0／本体 1.0**	**5 回中 5 回成功**	**ブロックが指の間で安定して保持された**

3. システム構成と手順

Gazebo Harmonic（ヘッドレス・GPU 描画）　SO-101＋2本指グリッパ・剛体ブロック・トレイ・カメラ×2

▲ 関節目標・指の開き幅　　**ros_gz_bridge**　　▼ 画像 160×120・関節状態

ROS 2 ノード（Python 3.12）　collect：IK の手本で操作して記録 ／ eval：観測を推論サーバへ送り、返ってきた行動を指令

データ化・学習 LeRobotDataset → lerobot-train（ACT, GPU）	⇄ **TCP**	**推論サーバ** 学習済み ACT（Python 3.13＋GPU）

1. **学習データの作成**：逆運動学による手本が「接近 → 下降 → 挟む → 保持 → 持ち上げ → 運搬 → 下降 → 開く → 帰還」を実行します。ブロックの位置と向き、置き位置、各区間の速さを毎回ランダムにしました。成功判定は、Gazebo から取得したブロックの実際の位置で行います（トレイ内に置かれ、指が開いていること）。65 回中 60 回成功し、成功分だけを保存しました。
2. **学習**：LeRobot の ACT を 10,000 ステップ（バッチ 32、約 70 分）学習しました。
3. **評価**：学習に使っていない乱数で 10 か所の配置を作り、自律実行しました。

4. モデル：ACT（Action Chunking with Transformers）

ACT は、人などのお手本の動きを真似して覚える**模倣学習**のモデルです。2023 年に、スタンフォード大学の両腕ロボット **ALOHA** の研究（Zhao et al., RSS 2023）で発表され、LeRobot に標準で入っています。画像を ResNet-18 で特徴に変換し、Transformer（と CVAE）で「この先の一連の動き」をまとめて予測するのが特徴です。

入力と出力

入力	① カメラ画像（作業台を**斜め上から見下ろす**カメラ、160×120 画素） ② 現在のロボットの状態 6 個（腕の関節角 5 個＋指の開き幅 1 個） ※ ブロックの位置は与えていません。ポリシーは画像から位置を読み取ります。
出力	この先 **20 ステップ（2 秒分）**の行動の列（各ステップ：腕の関節目標 5 個＋指の開き幅 1 個）。そのうち最初の **10 ステップ（1 秒分）**を実行します。

アクションチャンクの設定（第1版と同じ）

chunk_size = 20	ポリシーは 1 回の推論で、この先 **20 ステップ分**の行動（関節目標 5 つと指の開き幅）をまとめて出力します。
n_action_steps = 10	そのうち**最初の 10 ステップ分だけ**を実際に実行し、残りは捨てて、次の推論で新しく出し直します。
時間に直すと	制御は 10 Hz なので、2 秒先まで予測し、1 秒分を実行して、1 秒ごとに推論し直します。1 回の作業（約 10 秒）で約 10 回推論します。

先までまとめて予測すると、動きが滑らかになり迷いが減ります。一方で、実行を 1 秒分に抑えて推論し直すことで、新しいカメラ画像を見て修正できます。

5. 結果

5.1 学習

ステップ	800	1,000	1,200	6,600	8,000	9,200	10,000
損失（L1＋KL）	1.447	1.208	1.018	0.098	0.081	0.070	0.065

損失は第1版の最終値（0.066）とほぼ同じ水準まで下がりました。

5.2 自律実行（同じ 10 か所の配置で、推論のしかたを比較）

推論のしかた	成功	失敗の内訳
学習時の設定（10 ステップごとに推論し直す）	4 / 10	つかみ損ね 4、弾き飛ばし 2
5 ステップごとに推論し直す	3 / 10	つかみ損ね 7
時間的アンサンブル（毎ステップ推論し、重なった予測を平均）	**5 / 10**	**つかみ損ね 5（弾き飛ばしはなし）**

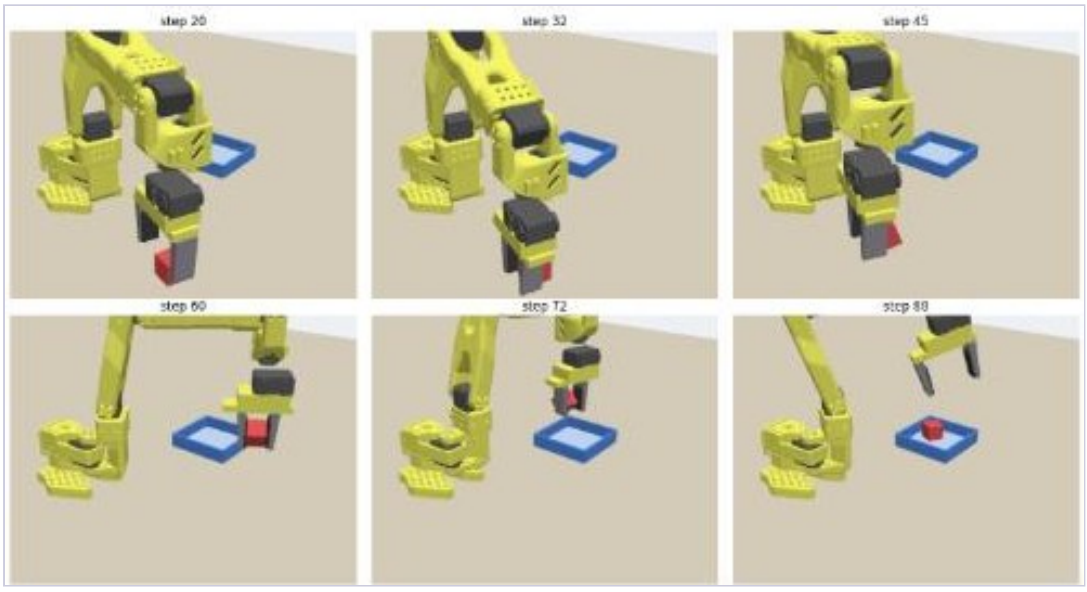


図3　成功例（時間的アンサンブル）：近づく（20）→ 両指で挟む（32）→ 挟んだまま持ち上げる（45）→ トレイ上へ（60）→ 開いて置く（72）→ 帰還（88）。数字はステップ（0.1 秒単位）

6. 工夫した点（評価の観点との対応）

観点	工夫した点
創造性	市販の SO-101 の片側ジョーを、両側が開閉する平行2本指グリッパに作り替えた。さらに、指の内側にラバーパッドを付け、指の本体を 10 段階のなだらかな形状にするなど、把持に向いたハンドを自分で設計した。
技術力	・URDF を直接書き換えてハンドを改造し、衝突（ビットマスク）と摩擦を部位ごとに設定した。 ・固定などの補助を使わず、力の上限付き位置制御で挟み、**摩擦力だけ**で物体を保持させた。 ・ブロックの実際の位置を各段階で計測する診断ツールを作り、「滑り」「手首の可動範囲」「IK の局所解」を一つずつ特定して直した（手本の成功率 1/3 → 5/5）。 ・ROS 2（Python 3.12）と LeRobot（Python 3.13・GPU）を TCP でつなぎ、推論のしかた（再推論の間隔・時間的アンサンブル）を学習し直さずに切り替えて比較できるようにした。
実用性	実際のロボットハンドと同じく「挟んで摩擦で持つ」方式で学習・評価しており、実機への応用を意識した。カメラ画像と関節状態だけで動き、物体の位置を事前に知らなくてよい。
表現力	指の断面図、把持の調整の経過、推論設定の比較表、成功例のコマ送りで、設計の意図と結果を分かりやすく示した。うまくいかなかった点も含めて正直に記録した。

7. 考察

- **摩擦把持は難しい**：成功率は第1版（固定方式、9/10）より大きく下がりました。摩擦だけで挟む方式では、手先の位置や指を閉じるタイミングがわずかにずれるだけで失敗します。人や実機のロボットが小物をつかむときと同じ難しさが、そのまま現れました。
- **失敗の主な原因はつかみ損ね**：失敗の大半は、ブロックが最初の位置の近くに残るものでした。手本 60 本では、場所ごとの「どこで止まって閉じるか」を覚えるには例が足りないと考えられます。
- **時間的アンサンブルの効果**：予測を平均することで動きが滑らかになり、ブロックを弾き飛ばす失敗がなくなって、成功率が最も高くなりました。
- **ラバーパッドの効果**：手本の動作で、ブロックが指の間で滑る現象がなくなりました（5/5）。摩擦把持では、指の表面の性質が重要であることを確認できました。

今後の改善

- 手本を 60 本から 120 本程度に増やす。つかみ損ねたときにやり直す動きも手本に含める。
- 画像の水増し（明るさ・位置のゆらぎ）や、学習時からの時間的アンサンブルの利用。

開発中の反省：Colab のランタイムが時間切れで切断された際、学習済みモデルを保存するコマンドが実体ではなくショートカット（シンボリックリンク）だけを保存していたため、モデルと評価映像が失われました。ノートブックは修正済みです（実体ごと保存、評価映像も保存）。

8. まとめ

SO-101 のハンドを平行2本指グリッパに作り替え、ラバーパッドと 10 段階のなだらかな指形状にしました。その結果、剛体のブロックを**摩擦力だけで**挟んで運ぶ手本動作を安定させました。これを ACT で模倣学習させたところ、画像と関節状態だけから自律的に「挟む・運ぶ・置く」を実行でき、最良の設定で 10 回中 5 回成

功しました。第1版より成功率は下がりましたが、補助なしの摩擦把持という、より現実に近い難しさに取り組み、課題と改善の方向をはっきりさせることができました。

参考

- SO-101 モデル：TheRobotStudio/SO-ARM100（Apache-2.0）。グリッパ部分は本課題で改造
- T. Z. Zhao et al., "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware", RSS 2023（ACT・ALOHA）
- LeRobot（Hugging Face）、ROS 2 Jazzy、Gazebo Harmonic、ros_gz